Spring AOP Internals

Chapter 03 -- Spring Deep Dive Tutorial

Aspect-Oriented Programming from Concept to Bytecode

A. Cross-Cutting Concerns	The scattered-code problem AOP solves
B. AOP Terminology	Aspect, Advice, Pointcut, JoinPoint, Weaving
C. @Before Advice	Interception mechanics and call stack
D. @After / @AfterReturning / @AfterThrowing	Differences and use cases
E. @Around Advice	ProceedingJoinPoint and full control
F. Pointcut Expressions	execution(), within(), @annotation()
G. JDK vs CGLIB Proxy	When Spring picks each and why
H. Self-Invocation Problem	Why @Transactional fails on internal calls
I. Real Logging Aspect	Execution time measurement end-to-end

A The Cross-Cutting Concern Problem

Spring AOP Internals

1. Plain English

In enterprise applications, certain behaviours -- logging, security checks, transaction management, and auditing -- must happen around many different methods in many different classes. If you implement each behaviour inside every method that needs it, you end up with duplicated boilerplate that is hard to maintain. Changing the logging framework forces edits across dozens of files. This is a **cross-cutting concern**: behaviour that cuts across module boundaries.

2. Real-World Analogy

Think of security badge scanners in an office building. Every room (OrderService, UserService, PaymentService) could install its own lock and reader -- the naive approach. Instead, a building-wide access-control system (an Aspect) intercepts every entry attempt at the doorframe (the proxy), checks the badge, and only then unlocks the door (calls the real method). One system, many rooms.

3. Minimal Code -- The Problem Without AOP

```
public class OrderService {
    public void placeOrder(Order o) {
        log.info("Entering placeOrder");    // duplicated in every service
        authCheck();                        // duplicated in every service
        txManager.begin();                  // duplicated in every service
        // --- real business logic ---
        orderRepo.save(o);
        // --------------------------------
        txManager.commit();                 // duplicated in every service
        log.info("Exiting placeOrder");     // duplicated in every service
    }
}
```

4. Diagram

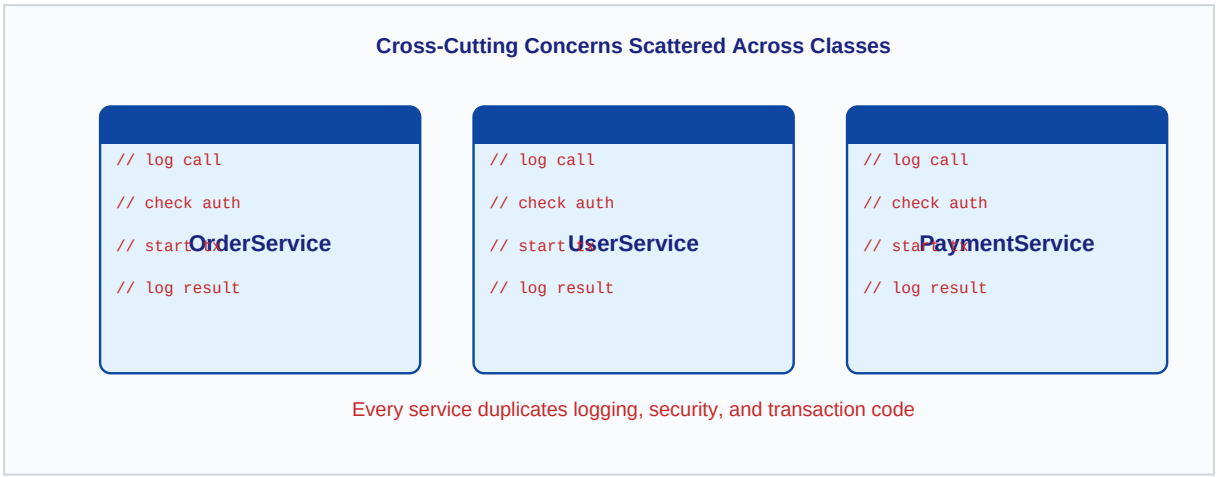

Figure A-1: Logging, auth and TX code duplicated in every service class

5. Behind the Scenes

Spring solves this with Aspect-Oriented Programming (AOP). When the container starts, **AnnotationAwareAspectJAutoProxyCreator** -- a BeanPostProcessor registered by

@EnableAspectJAutoProxy -- inspects every bean. For beans whose methods match a pointcut, it replaces the bean reference in the context with a **proxy object**. The proxy wraps the real bean and executes advice before, after, or around the real method. Callers always talk to the proxy, never the original bean.

6. Realistic Practical Example

A payment gateway integration requires logging every request/response for PCI-DSS compliance, authentication for every sensitive call, and transaction management. Without AOP, a single PaymentService might have 200 lines of cross-cutting code out of 250 total. With AOP, those 200 lines move to three focused Aspect classes and PaymentService shrinks to 50 lines of pure business logic.

7. Common Mistakes

Key Rules

- Thinking AOP is magic -- it is just proxy objects and method interception.
- Applying AOP to every method 'just in case' -- pointless overhead on non-shared concerns.
- Forgetting that AOP only works on Spring-managed beans -- new MyService() bypasses it entirely.
- Confusing cross-cutting concerns with regular shared utilities -- use a plain library for those.

B AOP Terminology

Spring AOP Internals

1. Plain English

AOP introduces a small vocabulary. Understanding each term precisely prevents confusion when reading stack traces or writing pointcut expressions.

Aspect: A class annotated @Aspect that bundles related advice. Example: LoggingAspect.

Advice: The actual code that runs at an interception point. Annotations: @Before, @After, @Around, etc.

JoinPoint: A specific execution point -- in Spring AOP always a method execution.

Pointcut: A predicate (expression) that matches a set of JoinPoints. Example: execution(* com.app.service.*.*(..))

Weaving: Applying aspects to target objects. Spring does this at container startup (runtime weaving via proxies).

Target: The original bean whose method is being advised. The proxy delegates to this object.

Proxy: The Spring-generated wrapper placed in front of the Target. Callers talk to the proxy.

2. Real-World Analogy

Imagine a hotel concierge desk. The guest (Caller) never walks directly to the kitchen (Target). The concierge (Proxy) intercepts the request. The concierge rulebook (Aspect) defines what to do: check reservation (Advice) only when the guest asks for a table (Pointcut matching that JoinPoint). Training the concierge from the rulebook is Weaving.

3. Minimal Code

```
@Aspect          // This class is an Aspect
@Component
public class AuditAspect {

    // Pointcut: matches all public methods in the service package
    @Pointcut("execution(public * com.app.service.*.*(..))")
    public void serviceLayer() {}

    // Advice: runs before any matched JoinPoint
    @Before("serviceLayer()")
    public void logBefore(JoinPoint jp) {  // jp = the JoinPoint
        System.out.println("Calling: " + jp.getSignature().getName());
    }
}
```

4. Diagram

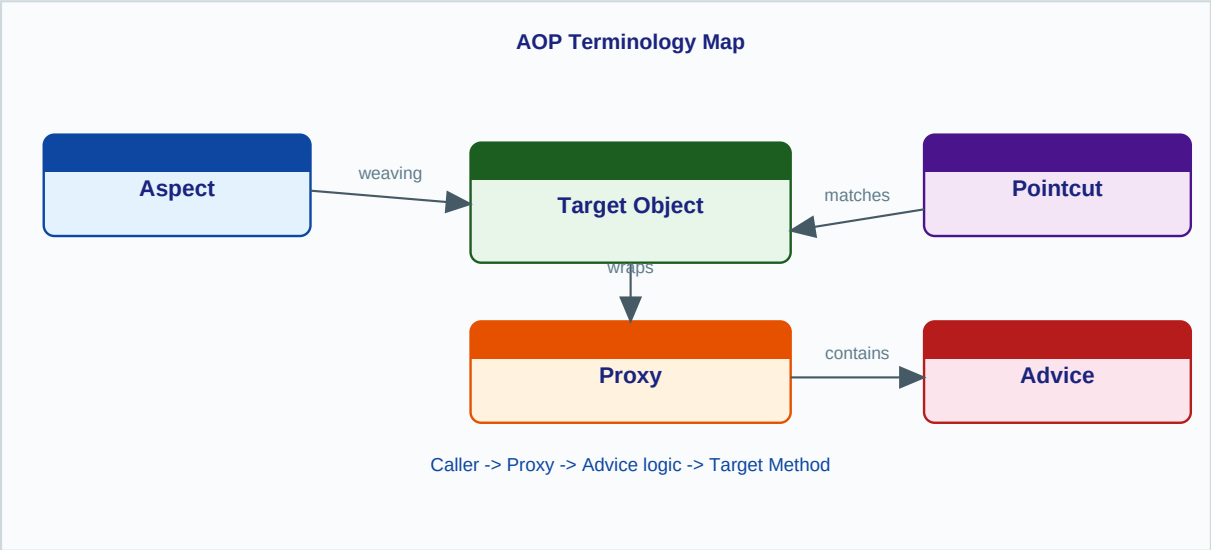

Figure B-1: Relationships between Aspect, Advice, Pointcut, JoinPoint, Proxy and Target

5. Behind the Scenes

At startup, **AnnotationAwareAspectJAutoProxyCreator.postProcessAfterInitialization()** is called for every bean. It uses **AspectJExpressionPointcut** to check whether any advice pointcut matches any method of the bean. If yes, it creates a proxy (JDK or CGLIB) and substitutes it in the application context. Weaving happens once at startup -- no per-call reflection overhead beyond the proxy dispatch itself.

6. Practical Example

A security aspect defines pointcut @annotation(Secured) -- matching every method annotated with your custom @Secured. The Aspect (SecurityAspect) contains a @Before advice that reads the annotation's role attribute and checks the current principal. One Aspect, zero changes to service classes.

7. Common Mistakes

Key Rules

- Confusing JoinPoint (a specific execution) with Pointcut (the matching expression).
- Calling the target object the 'proxy' -- the proxy is the generated wrapper; target is the original.
- Thinking 'weaving' only happens at class-load time -- Spring uses runtime proxy-based weaving.
- Forgetting to add @Component to @Aspect classes -- without it Spring never processes them.

C @Before Advice

Spring AOP Internals

1. Plain English

@Before advice runs *before* the target method executes. It receives a JoinPoint object with metadata (method name, arguments, target object). It cannot stop execution unless it throws an exception. It has no access to the return value because the target has not yet run.

2. Real-World Analogy

A nightclub bouncer standing at the door is @Before advice. You must pass the bouncer (advice runs) before you enter the club (target method). The bouncer can throw you out (throw an exception to abort), but cannot change what happens inside the club.

3. Minimal Code

```
@Aspect
@Component
public class LoggingAspect {

    @Before("execution(* com.app.service.*.*(..))")
    public void logMethodEntry(JoinPoint jp) {
        String method = jp.getSignature().toShortString();
        Object[] args = jp.getArgs();
        log.info("[BEFORE] {} called with {} args", method, args.length);
        // Throwing here would abort the target call entirely
    }
}
```

4. Diagram

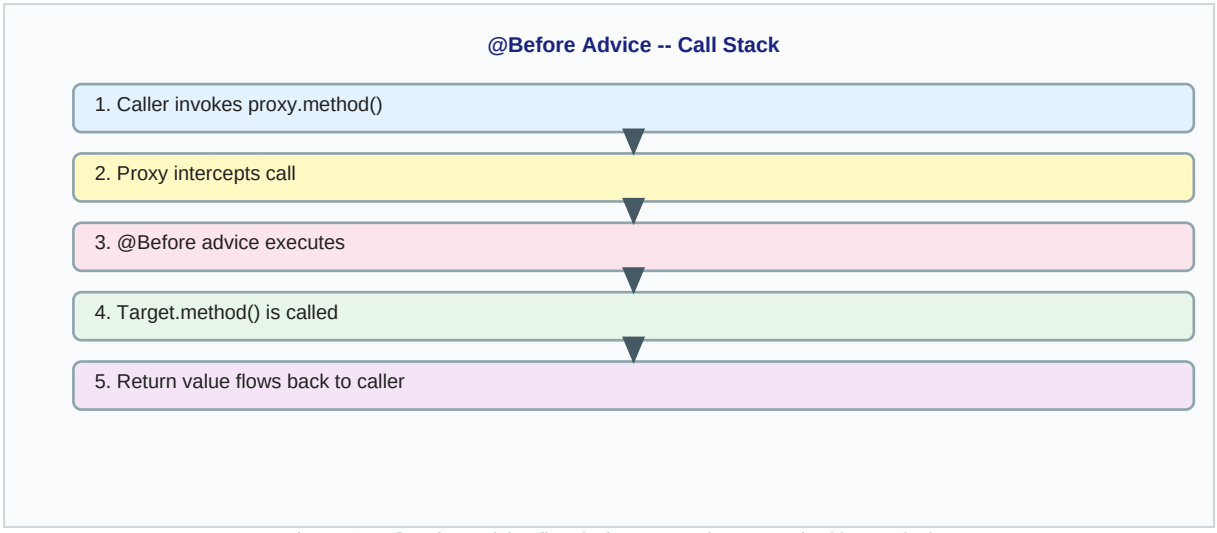

Figure C-1: @Before advice fires before step 4 (target method invocation)

5. Behind the Scenes

When the proxy method is invoked, Spring's **ReflectiveMethodInvocation.proceed()** walks through an ordered chain of MethodInterceptor instances. For @Before advice, Spring wraps it in a **MethodBeforeAdviceInterceptor** which calls **advice.before(method, args, target)** and then calls **mi.proceed()** to continue the chain. The JoinPoint passed to your method is a

MethodInvocationProceedingJoinPoint backed by the ReflectiveMethodInvocation.

6. Practical Example -- Argument Binding

```
@Before("execution(* com.bank.service.*.transfer(..)) && args(amount, toAccount)")
public void auditTransfer(JoinPoint jp, BigDecimal amount, String toAccount) {
    auditLog.record("Transfer attempt", amount, toAccount,
        SecurityContextHolder.getContext().getAuthentication().getName());
    // Spring binds 'amount' and 'toAccount' from method parameters automatically
}
```

7. Common Mistakes

Key Rules

- Expecting to read the return value in @Before -- impossible, the method has not run yet.
- Assuming @Before can alter the method arguments -- JoinPoint exposes them read-only.
- Forgetting that throwing from @Before prevents the target from executing at all.
- Using jp.getArgs() and casting blindly without checking argument types at runtime.

D @After, @AfterReturning, @AfterThrowing

Spring AOP Internals

1. Plain English

Three advice annotations fire *after* the target method:

@After -- like a finally block. Runs whether the method returned normally or threw. Cannot access the return value or the exception.

@AfterReturning -- runs only on normal return. You can bind the return value with the *returning* attribute and inspect or log it.

@AfterThrowing -- runs only when an exception propagates. You can bind the exception with the *throwing* attribute. It cannot suppress the exception (use @Around for that).

2. Real-World Analogy

@After is the exit turnstile that counts everyone leaving regardless. @AfterReturning is the store receipt printer -- it only prints when the transaction succeeded. @AfterThrowing is the fire-exit alarm -- it only triggers when something went wrong.

3. Minimal Code

```
@Aspect @Component
public class ResultAspect {

    // Runs always -- like finally
    @After("execution(* com.app.service.*.*(..))")
    public void cleanup(JoinPoint jp) {
        MDC.clear(); // always release log context
    }

    // Runs only on normal return; 'result' is bound to the actual return value
    @AfterReturning(pointcut = "execution(* com.app.service.*.*(..))",
                    returning  = "result")
    public void logResult(JoinPoint jp, Object result) {
        log.info("{} returned: {}", jp.getSignature().getName(), result);
    }

    // Runs only on exception; 'ex' is bound to the thrown exception
    @AfterThrowing(pointcut = "execution(* com.app.service.*.*(..))",
                   throwing  = "ex")
    public void logException(JoinPoint jp, Exception ex) {
        log.error("{} threw: {}", jp.getSignature().getName(), ex.getMessage());
        // Cannot suppress the exception here -- use @Around for that
    }
}
```

4. Diagram

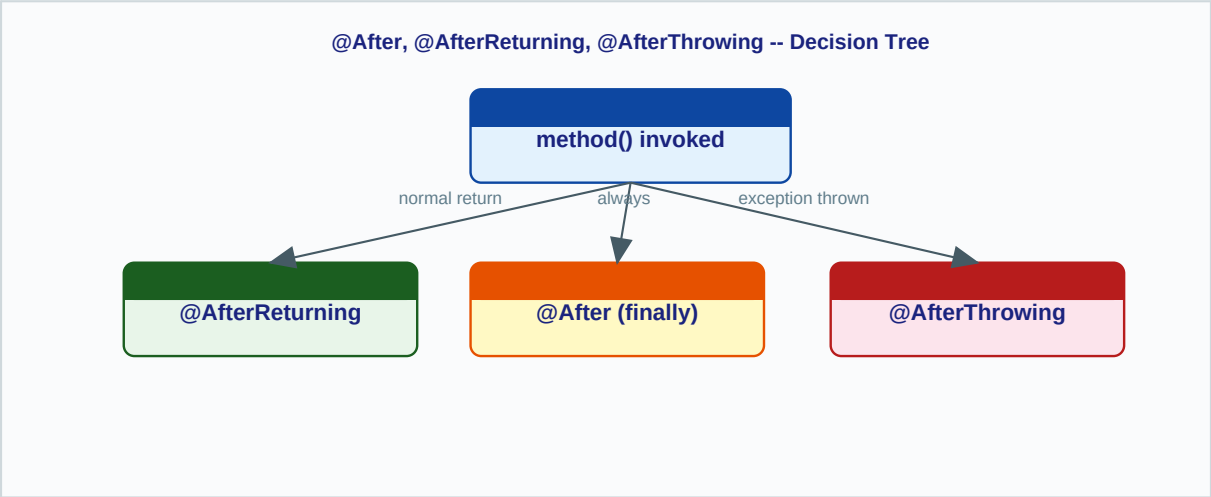

Figure D-1: Decision tree -- which post-method advice fires in each scenario

5. Behind the Scenes

@After is implemented as **AspectJAfterAdvice** which wraps the chain in a try/finally. @AfterReturning is **AfterReturningAdviceInterceptor** -- it calls mi.proceed() and on normal return invokes the advice with the return value. @AfterThrowing is **AspectJAfterThrowingAdvice** -- it calls mi.proceed() in a try/catch, invokes advice only in the catch block, then rethrows. All three are chained in a ReflectiveMethodInvocation interceptor list ordered by @Order or the Ordered interface.

6. Practical Example

```
// Emit a metric only when a service call succeeds
@AfterReturning("execution(* com.shop.OrderService.placeOrder(..))")
public void recordOrderPlaced() {
    meterRegistry.counter("orders.placed").increment();
}

// Page on-call if PaymentService throws a runtime exception
@AfterThrowing(pointcut = "execution(* com.shop.PaymentService.*(..))",
               throwing  = "ex")
public void alertOnPaymentFailure(RuntimeException ex) {
    alertService.sendPage("Payment failure: " + ex.getMessage());
}
```

7. Common Mistakes

Key Rules

- Using @AfterThrowing expecting to suppress the exception -- it cannot; only @Around can.
- Using @After to read the return value -- it has no access; use @AfterReturning.
- Declaring the 'returning' type too specifically -- if the type does not match, advice silently skips.
- Forgetting that @After runs even when the @Before advice threw -- the chain still unwinds.

E @Around Advice -- Full Control

Spring AOP Internals

1. Plain English

@Around is the most powerful advice. Your method *replaces* the normal invocation -- you decide when (or whether) to call the target by invoking **pjp.proceed()**. You can modify arguments before proceeding, modify the return value after, catch and swallow exceptions, retry the call, or add precise timing.

2. Real-World Analogy

@Around is like a relay race. The first runner (before-part of your advice) runs, then hands the baton (pjp.proceed()) to the second runner (target method). The first runner resumes when the baton returns (after-part) and can inspect or modify it. If needed, the first runner can run the second segment again (retry logic).

3. Minimal Code

```
@Aspect @Component
public class TimingAspect {

    @Around("execution(* com.app.service.*.*(..))")
    public Object measureTime(ProceedingJoinPoint pjp) throws Throwable {
        long start = System.nanoTime();
        try {
            Object result = pjp.proceed();     // invoke the real method
            long ms = (System.nanoTime() - start) / 1_000_000;
            log.info("{} took {} ms",
                pjp.getSignature().getName(), ms);
            return result;                     // MUST return the result
        } catch (Throwable ex) {
            log.error("Method failed: {}", ex.getMessage());
            throw ex;                          // rethrow or handle
        }
    }
}
```

4. Diagram

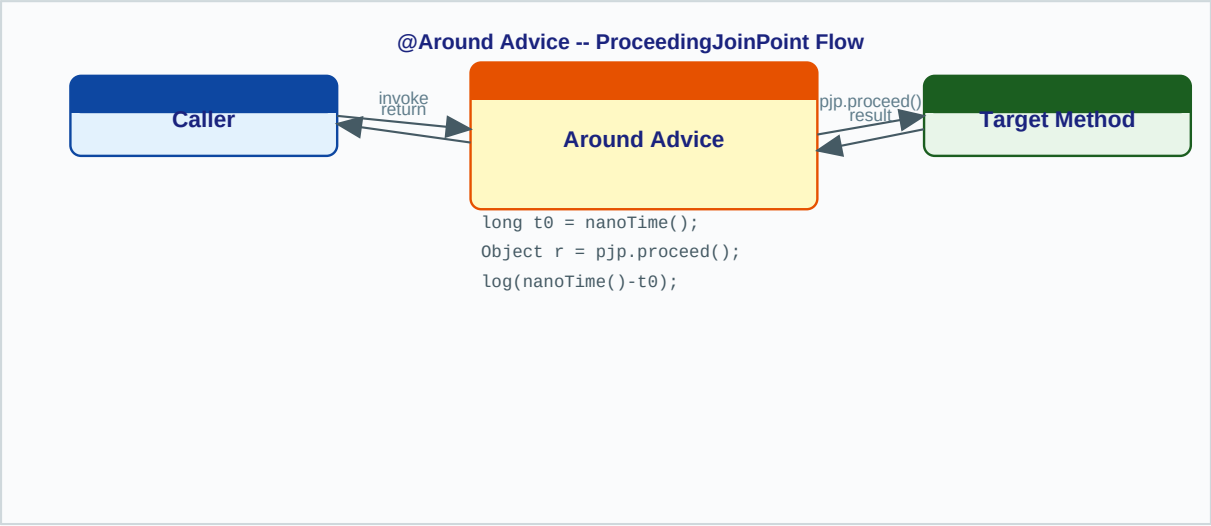

Figure E-1: @Around wraps the target entirely -- pjp.proceed() is the hand-off point

5. Behind the Scenes

@Around advice is wrapped in **AspectJAroundAdvice** which implements MethodInterceptor. The ProceedingJoinPoint is a **MethodInvocationProceedingJoinPoint** wrapping the same ReflectiveMethodInvocation. Calling **pjp.proceed()** advances to the next interceptor in the chain (which may be another advice, or ultimately the target invocation via **AopUtils.invokeJoinpointUsingReflection()**). You can call pjp.proceed() zero times (skip target), once (normal), or multiple times (retry).

6. Practical Example -- Retry on Transient Failure

```
@Around("@annotation(retryable)")
public Object retry(ProceedingJoinPoint pjp, Retryable retryable) throws Throwable {
    int maxAttempts = retryable.maxAttempts();
    Throwable lastEx = null;
    for (int attempt = 1; attempt &lt;= maxAttempts; attempt++) {
        try {
            return pjp.proceed();
        } catch (TransientDataAccessException ex) {
            lastEx = ex;
            log.warn("Attempt {}/{} failed, retrying...", attempt, maxAttempts);
            Thread.sleep(200L * attempt);
        }
    }
    throw lastEx;
}
```

7. Common Mistakes

Key Rules

- Forgetting to return pjp.proceed() result -- non-void methods return null to callers silently.
- Not declaring 'throws Throwable' -- pjp.proceed() is declared to throw Throwable.
- Calling pjp.proceed() twice accidentally -- causes duplicate DB writes or duplicate emails.
- Using @Around when @Before or @AfterReturning would suffice -- prefer the simplest advice type.

F Pointcut Expressions

Spring AOP Internals

1. Plain English

A pointcut expression is a pattern that Spring evaluates against every method in every Spring bean to decide whether advice should run there. Spring uses the AspectJ pointcut language. The three most useful designators are **execution()**, **within()**, and **@annotation()**.

2. Real-World Analogy

A pointcut is like a search-engine query filter. execution() is a full-text search on method signatures. within() filters by package (like filtering by folder). @annotation() filters by a tag you put on the item. You combine them with &&, ||, and ! just like search operators.

3. Syntax Reference

Designator	Syntax Example	What It Matches
execution()	execution(public * com.app.service.*.*(..))	Method executions matching the full signature pattern
within()	within(com.app.service.*)	All methods inside the specified type or package
@annotation()	@annotation(com.app.Secured)	Methods annotated with the specified annotation
args()	args(java.lang.String, ..)	Methods whose runtime arguments match the given types
bean()	bean(orderService)	Methods on a bean with the given name (Spring extension)
&& || !	execution(..) && @annotation(..)	Combine multiple designators with logical operators

execution() Pattern Breakdown

```
// execution( [modifiers] return-type [class-pattern].method-name(params) )
//
// *   = any single type or package segment
// ..  = any number of packages (type pattern) OR any arguments (params)
// +   = this type and all subclasses

execution(public * com.app.service.*.*(..))
//              ^  ^              ^ ^  ^^
//              |  return-type    | |  any args
//              modifier          | any method name
//                                class wildcard within service package

execution(* com.app..*(..))          // any class in com.app or subpackages
execution(* *Service.find*(String))  // methods starting 'find', one String arg
```

4. Diagram -- Pointcut Matching at Proxy Creation Time

```
Spring container startup:
  for each bean in context:
    for each method in bean class:
      AspectJExpressionPointcut.matches(method, targetClass)
        -> if true: register MethodInterceptor in proxy chain for this method
  Result: proxy has a pre-built interceptor chain per matching method
  (No re-evaluation at call time -- cached as ShadowMatch per method signature)
```

5. Behind the Scenes

AspectJExpressionPointcut delegates to the AspectJ weaver's PointcutParser and ShadowMatch infrastructure to evaluate expressions. Spring caches the ShadowMatch result per (method, targetClass) pair so the expression is evaluated only once per method signature, not on every call. At call time, ReflectiveMethodInvocation simply executes the cached interceptor chain with no further pattern matching.

6. Practical Example -- Composing Pointcuts

```
@Aspect @Component
public class AuditAspect {

    @Pointcut("execution(public * com.bank.service.*.*(..))")
    public void bankServiceOps() {}

    @Pointcut("@annotation(com.bank.Audited)")
    public void auditedMethods() {}

    // Intersection: public service methods that are also @Audited
    @Pointcut("bankServiceOps() &amp;&amp; auditedMethods()")
    public void auditedServiceOps() {}

    @Before("auditedServiceOps()")
    public void audit(JoinPoint jp) {
        auditLog.write(jp.getSignature().toShortString());
    }
}
```

7. Common Mistakes

Key Rules

- Using com.app.service.* (single dot) instead of com.app.service..* (double dot) for subpackages.
- execution() matches the method signature, not the call site -- caller location is irrelevant.
- Writing execution(* *(..)) matching everything -- enormous overhead and unintended framework interception.
- Mismatching param types: execution(* *(Long)) only matches java.lang.Long, not primitive long.
- Forgetting that @annotation() is case-sensitive and requires the fully qualified class name.

G JDK Dynamic Proxy vs CGLIB Proxy

Spring AOP Internals

1. Plain English

Spring must create a proxy object that sits in front of your bean. It has two strategies:

JDK Dynamic Proxy -- uses java.lang.reflect.Proxy to create an object that implements the same interfaces as your bean. Works at runtime with no additional bytecode generation. Requires the target to implement at least one interface.

CGLIB -- generates a subclass of your bean class at runtime, overriding all non-final methods to insert the interceptor chain. No interface required. Spring has bundled CGLIB since Spring 3.2. Spring Boot 2.0+ enables CGLIB by default.

2. Real-World Analogy

JDK proxy is a contract interpreter: it only works if there is a signed contract (interface) to refer to. CGLIB is a method-actor understudy: it learns every move (method) of the original actor (class) and performs them -- plus inserts extra scenes (advice) -- with no contract required.

3. Minimal Code

```
// Force CGLIB for all proxies:
@EnableAspectJAutoProxy(proxyTargetClass = true)
@Configuration
public class AppConfig {}

// Spring Boot 2.0+ already defaults to CGLIB (proxyTargetClass = true)

// JDK proxy is used when:
//   - bean implements an interface
//   - AND proxyTargetClass = false (not the Spring Boot default)

// CGLIB is used when:
//   - bean has no interface, OR
//   - proxyTargetClass = true (Spring Boot default), OR
//   - bean is a @Configuration class (always CGLIB)
```

4. Diagram

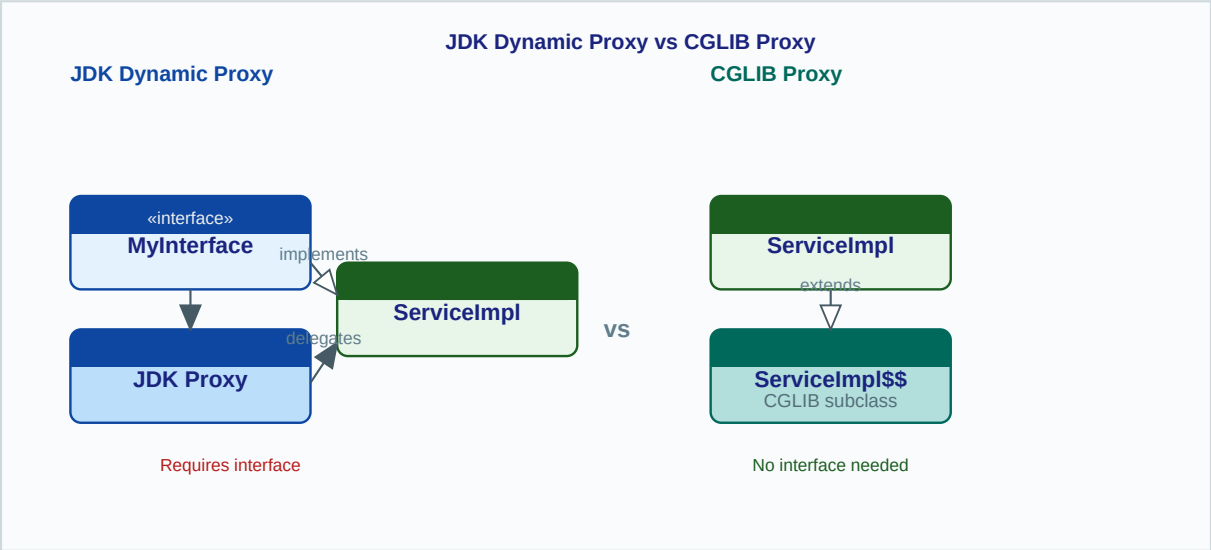

Figure G-1: JDK proxy requires interface; CGLIB generates a runtime subclass

5. Behind the Scenes

JDK Proxy: Spring calls java.lang.reflect.Proxy.newProxyInstance() with an InvocationHandler implemented by **JdkDynamicAopProxy**. Every method call on the proxy reaches **JdkDynamicAopProxy.invoke()** which builds a ReflectiveMethodInvocation and starts the interceptor chain.

CGLIB: Spring uses **ObjenesisCglibAopProxy** (backed by CGLIB Enhancer). It generates a subclass with overridden methods. Each overridden method calls **CglibAopProxy.DynamicAdvisedInterceptor.intercept()** which builds a CglibMethodInvocation and executes the interceptor chain. Spring uses **Objenesis** to instantiate the subclass without calling the parent constructor.

Decision Table

Factor	JDK Proxy	CGLIB
Requires interface?	Yes	No
Bytecode generation?	No (JDK built-in)	Yes (runtime subclass)
Final classes?	Can proxy (interface only)	Cannot subclass final classes
Final methods?	N/A	Cannot override -- advice silently skipped
Constructor called?	No	No (Objenesis bypasses it)
Spring Boot default?	No	Yes (proxyTargetClass=true since Boot 2.0)

6. Practical Example

If you inject OrderService by its concrete class type and Spring uses a JDK proxy, the injection fails with ClassCastException because the proxy only implements the interface, not the concrete class. Solution: inject by interface type, or switch to proxyTargetClass=true (already the Spring Boot default).

7. Common Mistakes

Key Rules

- Making a method or class final when using CGLIB -- Spring silently skips advice on final methods.
- Autowiring by concrete type with JDK proxy in use -- proxy is not a subclass of the concrete type.
- Assuming Spring Boot uses JDK proxies by default -- CGLIB has been the default since Spring Boot 2.0.
- Relying on the constructor running when using CGLIB -- Objenesis bypasses it entirely.

H Self-Invocation Problem

Spring AOP Internals

1. Plain English

Spring AOP works by replacing the bean reference in the context with a proxy. When external code calls a method, it goes through the proxy -- advice fires. But when a method inside the class calls another method on the same class using this.otherMethod(), it calls the raw target object, bypassing the proxy entirely. This means @Transactional, @Cacheable, @Async, and any custom advice on the inner method will silently not execute.

2. Real-World Analogy

The proxy is a telephone switchboard. Calls from outside the building go through the switchboard (advice fires). An employee calling a colleague on the same internal extension bypasses the switchboard entirely -- no call logging, no transfer rules apply. Self-invocation is the internal extension call.

3. Minimal Code -- The Bug and The Fix

```
// THE BUG
@Service
public class OrderService {
    public void placeOrder(Order o) {
        // this.validateOrder() calls the raw target, NOT the proxy
        // @Transactional on validateOrder() has NO effect here
        validateOrder(o);
        orderRepo.save(o);
    }

    @Transactional  // IGNORED when called via this.validateOrder() above
    public void validateOrder(Order o) { ... }
}

// FIX 1: Inject self (@Lazy avoids circular dependency)
@Service
public class OrderService {
    @Lazy @Autowired
    private OrderService self;  // this reference IS the proxy

    public void placeOrder(Order o) {
        self.validateOrder(o);  // goes through proxy -- @Transactional fires
    }

    @Transactional
    public void validateOrder(Order o) { ... }
}

// FIX 2: Extract validateOrder to a separate @Service bean (cleanest)
// FIX 3: AopContext.currentProxy() with exposeProxy=true on @EnableAspectJAutoProxy
```

4. Diagram

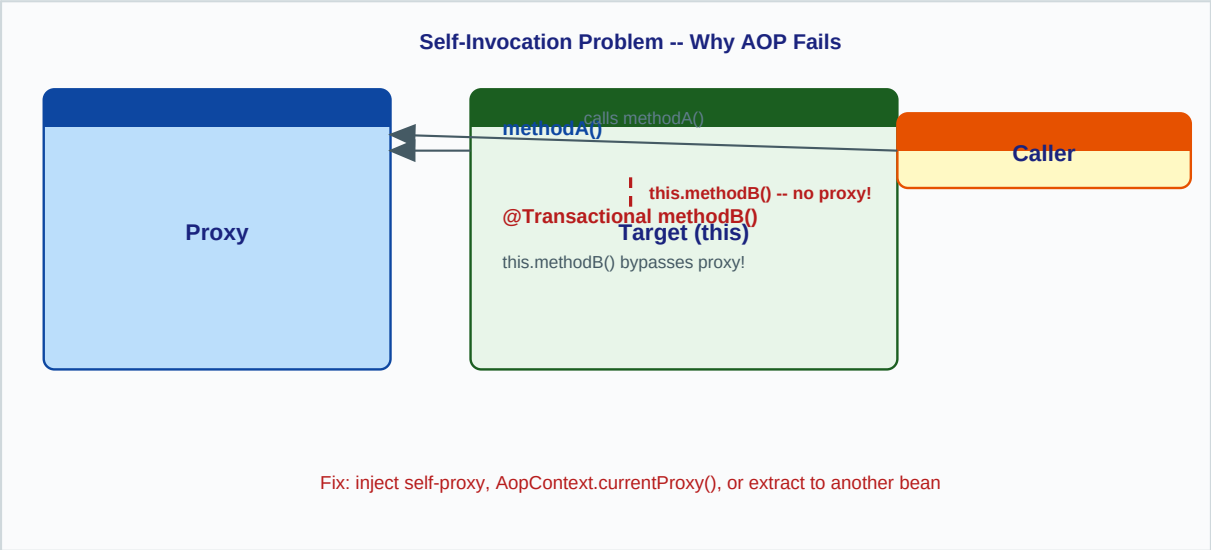

Figure H-1: this.methodB() calls the raw target directly, the proxy is never involved

5. Behind the Scenes

When the JVM executes this.validateOrder(o), *this* refers to the Java object currently executing -- the raw target, not the proxy. The proxy object is a separate Java object on the heap. The JVM has no mechanism to redirect a field reference to a different object. Spring has no way to intercept plain Java method calls; it can only intercept calls that go through the proxy reference held by other objects in the context.

AspectJ load-time weaving (LTW) or compile-time weaving do not have this limitation because they instrument bytecode directly inside the class. Spring AOP is proxy-based and therefore cannot overcome the self-invocation constraint.

6. Practical Example

A common real-world manifestation: placeOrder() is public; validateAndSave() is annotated @Cacheable. Calling validateAndSave() from inside the same class never caches. Teams spend hours debugging why the cache is never populated. The fix is always the same: make the call go through the proxy.

7. Common Mistakes

Key Rules

- Putting @Transactional on private methods -- Spring cannot proxy private methods.
- Using @Async on a method called from the same class -- the call runs synchronously.
- Thinking self-injection is a hack -- it is the correct Spring-idiomatic solution when extraction is not possible.
- Not setting exposeProxy=true before using AopContext.currentProxy() -- it throws IllegalStateException.

I Real Example: Logging Aspect + Execution Time

Spring AOP Internals

1. Plain English

We build a production-grade logging aspect that: (a) logs every service call entry with method name and arguments, (b) measures execution time in milliseconds, (c) logs the return value on success, and (d) logs the exception class on failure. It uses @Around for timing and MDC for correlation IDs.

2. Real-World Analogy

An airline flight recorder (black box) captures everything that happens during a flight without the pilots writing any logging code. Our aspect is the application's black box -- invisible to service developers but capturing everything for the operations team.

3. Complete Working Implementation

```
@Aspect
@Component
@Order(1)  // run before transaction aspects so we log inside the transaction
public class ExecutionLoggingAspect {

    private static final Logger log =
        LoggerFactory.getLogger(ExecutionLoggingAspect.class);

    @Pointcut("execution(public * com.app.service..*.*(..))")
    public void servicePublicMethods() {}

    @Around("servicePublicMethods()")
    public Object logExecution(ProceedingJoinPoint pjp) throws Throwable {
        String method  = pjp.getSignature().toShortString();
        Object[] args  = pjp.getArgs();
        String corrId  = UUID.randomUUID().toString().substring(0, 8);

        MDC.put("correlationId", corrId);
        log.info("[ENTER] {} args={}", method, Arrays.toString(args));

        long start = System.nanoTime();
        try {
            Object result = pjp.proceed();
            long ms = (System.nanoTime() - start) / 1_000_000;
            log.info("[EXIT] {} returned={} elapsed={}ms",
                method, summarize(result), ms);
            return result;
        } catch (Throwable ex) {
            long ms = (System.nanoTime() - start) / 1_000_000;
            log.error("[ERROR] {} threw={} elapsed={}ms",
                method, ex.getClass().getSimpleName(), ms);
            throw ex;
        } finally {
            MDC.remove("correlationId");
        }
    }

    private String summarize(Object obj) {
        if (obj == null) return "null";
        if (obj instanceof Collection)
            return "Collection[" + ((Collection) obj).size() + "]";
```

```
        String s = obj.toString();
        return s.length() > 80 ? s.substring(0, 80) + "..." : s;
    }
}
```

4. Diagram

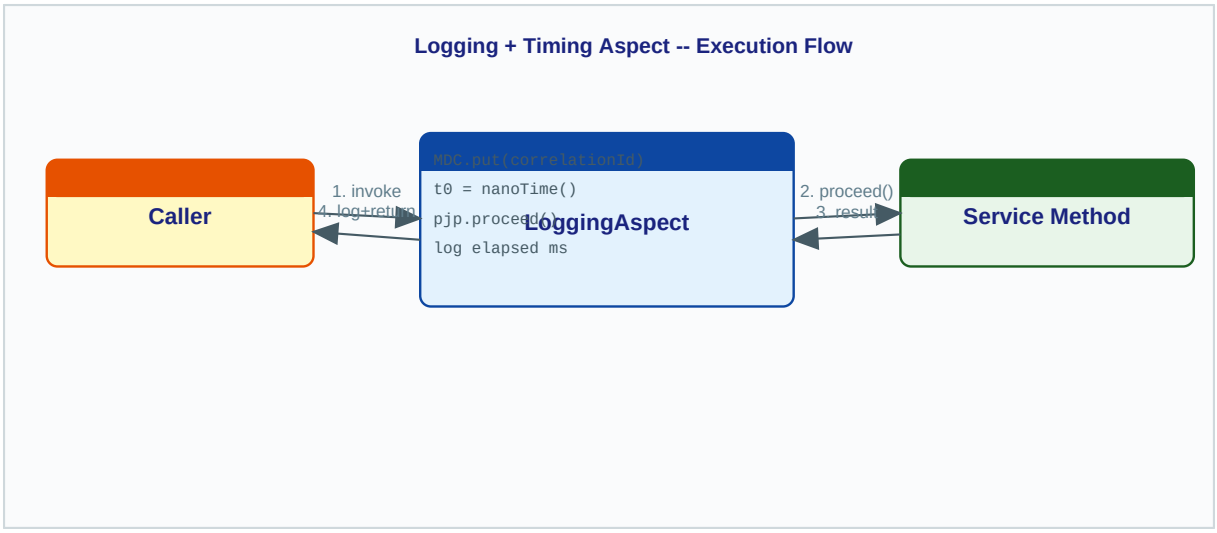

Figure I-1: Logging aspect wraps every service call -- entry log, timing, exit or error log

5. Behind the Scenes -- Full Startup-to-Call Sequence

1. Container startup: AnnotationAwareAspectJAutoProxyCreator scans all @Aspect beans. For ExecutionLoggingAspect it finds the @Around advice and its pointcut expression.

2. For each service bean (e.g., OrderService), AspectJExpressionPointcut.matches() returns true for all public methods. Spring creates a CGLIB subclass (Spring Boot default) wrapping OrderService and replaces the bean reference in the context.

3. On each method call, the CGLIB-generated override calls DynamicAdvisedInterceptor.intercept() which constructs a CglibMethodInvocation and runs the interceptor chain.

4. The chain contains: [ExecutionLoggingAspect Around interceptor, target invocation]. Our logExecution() runs; pjp.proceed() advances the chain to invoke the real OrderService method via reflection.

5. The result or exception flows back up the chain. Our finally block cleans up MDC. The caller receives the result transparently.

6. Extending to Slow-Call Alerting

```
private static final long SLOW_THRESHOLD_MS = 500;

// Replace the exit log in the try block:
long ms = (System.nanoTime() - start) / 1_000_000;
if (ms > SLOW_THRESHOLD_MS) {
    log.warn("[SLOW] {} took {}ms (threshold={}ms)",
        method, ms, SLOW_THRESHOLD_MS);
    meterRegistry.counter("slow.methods",
        "method", method).increment();
} else {
    log.info("[EXIT] {} elapsed={}ms", method, ms);
}
```

7. Common Mistakes and Production Tips

Key Rules

• Logging full toString() of arguments -- may expose PII or cause NullPointerException; always summarize.

• Not using @Order -- if transaction advice and logging advice both match, order determines whether you log inside or outside the transaction boundary.

• Enabling DEBUG-level argument logging in production -- massive log volume; gate behind log.isDebugEnabled().

• Forgetting to rethrow exceptions from @Around -- callers receive null instead of the exception.

• Not removing MDC keys in finally -- MDC leaks across thread-pool threads in async or reactive apps.

Spring AOP -- Quick Reference Cheat Sheet

Advice Type Comparison

Advice	Runs When	Access Return?	Suppress Exception?	Method Signature
@Before	Before method	No	No (throw only)	void m(JoinPoint jp)
@After	Always (finally)	No	No	void m(JoinPoint jp)
@AfterReturning	Normal return only	Yes	No	void m(JoinPoint jp, Object result)
@AfterThrowing	Exception only	No	No	void m(JoinPoint jp, Exception ex)
@Around	Replaces invocation	Yes	Yes	Object m(ProceedingJoinPoint pjp)

Proxy Type Selection

Proxy Type	Trigger	Mechanism	Key Limitation
JDK Dynamic	Bean implements interface + proxyTargetClass=false	java.lang.reflect.Proxy + InvocationHandler (JdkDynamicAopProxy)	Must inject by interface type; fails on concrete-type autowiring
CGLIB	No interface OR proxyTargetClass=true (Spring Boot default)	Runtime subclass via CGLIB Enhancer + MethodInterceptor (ObjenesisCglibAopProxy)	Cannot proxy final classes or override final methods

Common Mistakes Quick Reference

Mistake	Symptom	Fix
Self-invocation	@Transactional / @Cacheable ignored on internal calls	Inject self-proxy (@Lazy) or extract to another bean
Final method with CGLIB	Advice silently not applied	Remove final modifier or use interface + JDK proxy
Autowire concrete type (JDK)	ClassCastException at startup	Autowire by interface type
Missing @Component on @Aspect	Aspect never applied to any bean	Add @Component or declare as @Bean in @Configuration
Overly broad pointcut	Framework beans intercepted; startup errors	Restrict pointcut to your own packages
Forget to return pjp.proceed()	Non-void methods return null to callers	Always return the result of pjp.proceed()

Key Internal Spring Classes

Class	Role
AnnotationAwareAspectJAutoProxyCreator	BeanPostProcessor registered by @EnableAspectJAutoProxy; scans @Aspect beans and creates proxies
AspectJExpressionPointcut	Evaluates AspectJ pointcut expressions; caches ShadowMatch results per method signature
ReflectiveMethodInvocation	Walks the MethodInterceptor chain on each proxied method call
JdkDynamicAopProxy	InvocationHandler for JDK proxy; builds ReflectiveMethodInvocation on each invoke()
ObjenesisCglibAopProxy	CGLIB DynamicAdvisedInterceptor; builds CglibMethodInvocation on each intercept()
MethodBeforeAdviceInterceptor	Wraps @Before advice into the interceptor chain
AspectJAroundAdvice	Wraps @Around advice; provides ProceedingJoinPoint to the advice method
AspectJAfterThrowingAdvice	Wraps @AfterThrowing advice in try/catch; rethrows after advice execution